```cpp
// 用途：时间分块统计某位置过去各时刻达到阈值的次数。
// 时间：总计 O(n+q √q log q)，q 为操作数。
// 空间：O(n+q)。
// 例题：https://www.luogu.com.cn/problem/P3863
// 题意：区间加可为负数；询问某位置在当前操作之前有多少个时刻的值不小于阈值。
// 思路：按位置扫描，把区间加拆成两个位置事件；用时间轴分块支持后缀加与历史时刻阈值计数。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100001;
const int MAXB = 501;

int n, m;
int arr[MAXN];

// op == 1 表示修改事件、位置x、时刻t、修改效果v、空缺
// op == 2 表示查询事件、位置x、时刻t、查询标准v、问题编号q
struct Event {
    int op, x, t, v, q;
};

bool EventCmp(Event &a, Event &b) {
    return a.x != b.x ? a.x < b.x : a.t < b.t;
}

Event event[MAXN << 2];
int cnte, cntq;

// tim[i] = v，表示在i号时间点，所有数字都增加v
long long tim[MAXN];
// 时间块内的所有值要排序，方便查询 >= v的数字个数
long long sortv[MAXN];

// 时间分块
int blen, bnum;
int bi[MAXN];
int bl[MAXB];
int br[MAXB];
long long lazy[MAXB];

// 每个查询的答案
int ans[MAXN];

void innerAdd(int l, int r, long long v) {
    // 仅边角块逐点修改，并重建这一块的有序副本。
    for (int i = l; i <= r; i++) {
        tim[i] += v;
    }
    for (int i = bl[bi[l]]; i <= br[bi[l]]; i++) {
        sortv[i] = tim[i];
    }
    sort(sortv + bl[bi[l]], sortv + br[bi[l]] + 1);
}

void add(int l, int r, long long v) {
    // 时间轴上的 [l,r] 整段加：边角重建，完整块只加 lazy。
    if (l > r) {
        return;
    }
    if (bi[l] == bi[r]) {
        innerAdd(l, r, v);
    } else {
        innerAdd(l, br[bi[l]], v);
        innerAdd(bl[bi[r]], r, v);
        for (int i = bi[l] + 1; i <= bi[r] - 1; i++) {
            lazy[i] += v;
        }
```

```
    }
}

//查询从l到r区间上大于v的数字的个数
int innerQuery(int l, int r, long long v) {
    v -= lazy[bi[l]];
    int ans = 0;
    for (int i = l; i <= r; i++) {
        if (tim[i] >= v) {
            ans++;
        }
    }
    return ans;
}

//查询i号块中大于等于v的数字的个数
int getCnt(int i, long long v) {
    v -= lazy[i];
    int l = bl[i], r = br[i], m, pos = br[i] + 1;
    while (l <= r) {
        m = (l + r) >> 1;
        if (sortv[m] >= v) {
            pos = m;
            r = m - 1;
        } else {
            l = m + 1;
        }
    }
    return br[i] - pos + 1;
}

int query(int l, int r, long long v) {
    if (l > r) {
        return 0;
    }
    int ans = 0;
    if (bi[l] == bi[r]) {
        ans = innerQuery(l, r, v);
    } else {
        ans += innerQuery(l, br[bi[l]], v);
        ans += innerQuery(bl[bi[r]], r, v);
        for (int i = bi[l] + 1; i <= bi[r] - 1; i++) {
            ans += getCnt(i, v);
        }
    }
    return ans;
}

void addChange(int x, int t, int v) {
    // 位置扫描到 x 时，时刻 t 及以后都应受这次增量影响。
    event[++cnte].op = 1;
    event[cnte].x = x;
    event[cnte].t = t;
    event[cnte].v = v;
}

void addQuery(int x, int t, int v) {
    event[++cnte].op = 2;
    event[cnte].x = x;
    event[cnte].t = t;
    event[cnte].v = v;
    event[cnte].q = ++cntq;
}

void prepare() {
    blen = (int)sqrt(m);
```

```
    bnum = (m + blen - 1) / blen;
    for (int i = 1; i <= m; i++) {
        bi[i] = (i - 1) / blen + 1;
    }
    for (int i = 1; i <= bnum; i++) {
        bl[i] = (i - 1) * blen + 1;
        br[i] = min(i * blen, m);
    }
    // 同位置的事件按时刻排，查询只读 [1,t-1]，不会计入当前秒。
    sort(event + 1, event + cnte + 1, EventCmp);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }
    m++;// 时间轴重新定义，1是初始时刻、2、3 ... m+1
    for (int t = 2, op, l, r, v, x; t <= m; t++) {
        cin >> op;
        if (op == 1) {
            cin >> l >> r >> v;
            // 区间加拆成位置差分：走到 l 生效，越过 r 抵消。
            addChange(l, t, v);
            addChange(r + 1, t, -v);
        } else {
            cin >> x >> v;
            addQuery(x, t, v);
        }
    }
    prepare();
    for (int i = 1; i <= cnte; i++) {
        if (event[i].op == 1) {
            add(event[i].t, m, event[i].v);
        } else {
            ans[event[i].q] = query(1, event[i].t - 1, 1LL * event[i].v - arr[event[i].x]);
        }
    }
    for (int i = 1; i <= cntq; i++) {
        cout << ans[i] << '\n';
    }
    return 0;
}
```

```cpp
// 用途：维护单点修改，并查询下标模 x 等于 y 的元素和。
// 时间：预处理 O(n√n)，单点修改 O(√n)；小模数查询 O(1)，大模数查询 O(n/x)。
// 空间：O(n)。
// 注意：查询参数需满足 x≥1、0≤y<x；数组下标从 1 开始。
// 例题：https://www.luogu.com.cn/problem/P3396
// 题意：单点改值，询问下标模 x 等于 y 的元素和。
// 思路：小模数维护各余数桶，大模数直接枚举匹配下标。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 150001;
const int MAXB = 401;

int n, m, blen;
int arr[MAXN];
long long dp[MAXB][MAXB];

// dp[x][y] 是所有下标 i%x=y 的 arr[i] 之和；只预存 x<=√n。
long long query(int x, int y) {
    if (x <= blen) {
        return dp[x][y];
    }
    // x 大时满足条件的下标很少，直接按步长 x 跳着枚举。
    long long ans = 0;
    for (int i = y; i <= n; i += x) {
        ans += arr[i];
    }
    return ans;
}

void update(int i, int v) {
    int delta = v - arr[i];
    arr[i] = v;
    // 每个小模数恰好有一个余数桶受到影响。
    for (int x = 1; x <= blen; x++) {
        dp[x][i % x] += delta;
    }
}

void prepare() {
    blen = (int)sqrt(n);
    // 建出每个小模数的全部余数桶。
    for (int x = 1; x <= blen; x++) {
        for (int i = 1; i <= n; i++) {
            dp[x][i % x] += arr[i];
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }
    prepare();
    char op;
    int x, y;
    for (int i = 1; i <= m; i++) {
        cin >> op >> x >> y;
        if (op == 'A') {
            cout << query(x, y) << '\n';
        } else {
            update(x, y);
        }
    }
```

```
67      return 0;
68  }
```

```
// 用途：离线求树上两点路径中不同数值的个数。
// 时间：约 O((n+q)√n)。
// 空间：O(n+q+V)。
// 例题：https://www.luogu.com.cn/problem/SP10707
// 题意：多次查询树上两点路径中的不同数值数目。
// 思路：欧拉序把路径转成区间，莫队移动时切换点出现状态并处理 LCA。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 40001;
const int MAXM = 100001;
const int MAXP = 20;

// 查询的参数，jobl、jobr、lca、id
// 如果是类型1，那么lca == 0，表示空缺
// 如果是类型2，那么lca > 0，查询结果需要补充这个单点信息
struct Query {
   int l, r, lca, id;
};

int n, m;
int color[MAXN];
int sorted[MAXN];
int cntv;

Query query[MAXM];

int head[MAXN];
int to[MAXN << 1];
int nxt[MAXN << 1];
int cntg;

// dep是深度
// seg是括号序
// st是节点开始序
// ed是节点结束序
// stjump是倍增表
// cntd是括号序列的长度
int dep[MAXN];
int seg[MAXN << 1];
int st[MAXN];
int ed[MAXN];
int stjump[MAXN][MAXP];
int cntd;

// 分块
int bi[MAXN << 1];

// 窗口信息
bool vis[MAXN];
int cnt[MAXN];
int kind;

int ans[MAXM];

void addEdge(int u, int v) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    head[u] = cntg;
}

int kth(int num) {
    int left = 1, right = cntv, mid, ret = 0;
    while (left <= right) {
        mid = (left + right) / 2;
        if (sorted[mid] <= num) {
            ret = mid;
```

```
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ret;
}

void dfs(int u, int fa) {
    dep[u] = dep[fa] + 1;
    seg[++cntd] = u;
    st[u] = cntd;
    stjump[u][0] = fa;
    for (int p = 1; p < MAXP; p++) {
        stjump[u][p] = stjump[stjump[u][p - 1]][p - 1];
    }
    for (int e = head[u], v; e > 0; e = nxt[e]) {
        v = to[e];
        if (v != fa) {
            dfs(v, u);
        }
    }
    seg[++cntd] = u;
    ed[u] = cntd;
}

int lca(int a, int b) {
    if (dep[a] < dep[b]) {
        swap(a, b);
    }
    for (int p = MAXP - 1; p >= 0; p--) {
        if (dep[stjump[a][p]] >= dep[b]) {
            a = stjump[a][p];
        }
    }
    if (a == b) {
        return a;
    }
    for (int p = MAXP - 1; p >= 0; p--) {
        if (stjump[a][p] != stjump[b][p]) {
            a = stjump[a][p];
            b = stjump[b][p];
        }
    }
    return stjump[a][0];
}

bool QueryCmp(Query &a, Query &b) {
    if (bi[a.l] != bi[b.l]) {
        return bi[a.l] < bi[b.l];
    }
    return a.r < b.r;
}

// 窗口不管是加入还是删除node
// 只要遇到node就翻转信息即可
void invert(int node) {
    int val = color[node];
    if (vis[node]) {
        if (--cnt[val] == 0) {
            kind--;
        }
    } else {
        if (++cnt[val] == 1) {
            kind++;
        }
```

```
    }
    vis[node] = !vis[node];
}

void compute() {
    int winl = 1, winr = 0;
    for (int i = 1; i <= m; i++) {
        int jobl = query[i].l;
        int jobr = query[i].r;
        int lca = query[i].lca;
        int id = query[i].id;
        while (winl > jobl) {
            invert(seg[--winl]);
        }
        while (winr < jobr) {
            invert(seg[++winr]);
        }
        while (winl < jobl) {
            invert(seg[winl++]);
        }
        while (winr > jobr) {
            invert(seg[winr--]);
        }
        // 注意这里  如果需要额外添加一个lca  结束的时候不要忘记删除它  防止污染后面的数据
        if (lca > 0) {
            invert(lca);
        }
        ans[id] = kind;
        if (lca > 0) {
            invert(lca);
        }
    }
}

void prepare() {
    for (int i = 1; i <= n; i++) {
        sorted[i] = color[i];
    }
    sort(sorted + 1, sorted + n + 1);
    cntv = 1;
    for (int i = 2; i <= n; i++) {
        if (sorted[cntv] != sorted[i]) {
            sorted[++cntv] = sorted[i];
        }
    }
    for (int i = 1; i <= n; i++) {
        color[i] = kth(color[i]);
    }
    // 括号序列分块
    int blen = (int) sqrt(cntd);
    for (int i = 1; i <= cntd; i++) {
        bi[i] = (i - 1) / blen + 1;
    }
    sort(query + 1, query + m + 1, QueryCmp);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> color[i];
    }
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        addEdge(u, v);
```

```
199             addEdge(v, u);
200         }
201         dfs(1, 0);
202         for (int i = 1, u, v, uvlca; i <= m; i++) {
203             cin >> u >> v;
204             if (st[v] < st[u]) {
205                 swap(u, v);
206             }
207             uvlca = lca(u, v);
208             if (u == uvlca) {
209                 query[i].l = st[u];
210                 query[i].r = st[v];
211                 query[i].lca = 0;
212             } else {
213                 query[i].l = ed[u];
214                 query[i].r = st[v];
215                 query[i].lca = uvlca;
216             }
217             query[i].id = i;
218         }
219         prepare();
220         compute();
221         for (int i = 1; i <= m; i++) {
222             cout << ans[i] << '\n';
223         }
224         return 0;
225     }
```

```cpp
// 用途：随机权重集合哈希，判断区间里每个值的出现次数是否都能被 k 整除，支持单点赋值。
// 时间：离散化 O((n+q)log(n+q))，T 轮检验 O(T(n+q)log n)，这里 T=20。
// 空间：O(n+q)。
// 注意：蒙特卡洛算法；NO 一定正确，YES 有极小误判概率；k≥1，n,q≤300000。
// 例题：https://www.luogu.com.cn/problem/CF1746F
// 题意：单点修改数组，询问某区间中每种数字的出现次数是否都是 k 的倍数。
// 思路：给每个值赋独立随机权重；合法区间的权重和必被 k 整除，重复多轮排除误判。
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 3e5+5;
const int MOD = 1e9+7;

int n,q,m=1;
int nums[MAXN];
int tmp[MAXN<<1];
bool ans[MAXN];
int cnt;
int h[MAXN<<1];

int op[MAXN];
int l[MAXN],r[MAXN],k[MAXN];
int pos[MAXN],v[MAXN];

int tree[MAXN];
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

inline int read(){
    int x=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9'){
        if(ch=='-')
            f=-1;
        ch=getchar();
    }
    while(ch>='0' && ch<='9')
        x=x*10+ch-'0',ch=getchar();
    return x*f;
}

inline int lowbit(int x){
    return x&-x;
}

void add(int x,int v){
    while(x<=n){
        tree[x]+=v;
        x+=lowbit(x);
    }
}

int query(int x){
    int ans=0;
    while(x){
        ans+=tree[x];
        x-=lowbit(x);
    }
    return ans;
}

int find(int val){
    int l=1,r=m,ans=1;
    while(l<=r){
        int mid=(l+r)>>1;
        if(tmp[mid]>=val){
            ans=mid;
```

```
            r=mid-1;
        }
        else{
            l=mid+1;
        }
    }
    return ans;
}

signed main()
{
    n=read(),q=read();
    for(int i=1;i<=n;i++){
        nums[i]=read();
        tmp[i]=nums[i];
    }
    cnt=n;
    for(int i=1;i<=q;i++){
        ans[i]=true;
        op[i]=read();
        if(op[i]==1){
            pos[i]=read(),v[i]=read();
            tmp[++cnt]=v[i];
        }
        else{
            l[i]=read(),r[i]=read(),k[i]=read();
        }
    }
    sort(tmp+1,tmp+cnt+1);
    for(int i=2;i<=cnt;i++){
        if(tmp[i]!=tmp[i-1]){
            tmp[++m]=tmp[i];
        }
    }
    for(int i=1;i<=n;i++){
        nums[i]=find(nums[i]);
    }
    for(int i=1;i<=q;i++){
        if(op[i]==1){
            v[i]=find(v[i]);
        }
    }
    for(int i=1;i<=n;i++){
        tmp[i]=nums[i];
    }

    int T=20;
    while(T--){
        for(int i=1;i<=n;i++){
            tree[i]=0;
            nums[i]=tmp[i];
        }
        // 每轮重新给离散后的每个值赋随机权重，误判概率随轮数下降。
        for(int i=1;i<=m;i++){
            h[i]=rng()%MOD;
        }
        for(int i=1;i<=n;i++){
            add(i,h[nums[i]]);
        }
        for(int i=1;i<=q;i++){
            if(op[i]==1){
                add(pos[i],h[v[i]]-h[nums[pos[i]]]);
                nums[pos[i]]=v[i];
            }
            else{
                if(ans[i]){
```

```
                    if((r[i]-l[i]+1)%k[i]){
                        ans[i]=false;
                    }
                    // 频次都为 k 的倍数时，权重和必为 k 的倍数；逆命题仅概率成立。
                    int all=query(r[i])-query(l[i]-1);
                    if(all%k[i]){
                        ans[i]=false;
                    }
                }
            }
        }
    }

    for(int i=1;i<=q;i++){
        if(op[i]==2){
            if(ans[i]){
                cout<<"YES"<<endl;
            }
            else{
                cout<<"NO"<<endl;
            }
        }
    }
    return 0;
}
```

```cpp
// 用途：指数以十进制大整数给出时，计算 a^b mod m。
// 时间：O(√m+|b|+log m)，|b| 是指数字符串长度。
// 空间：O(|b|)。
// 例题：https://www.luogu.com.cn/problem/P5091
// 题意：指数是十进制大整数，计算 a^b mod m。
// 思路：先求 φ(m)，读入大指数并根据扩展欧拉定理缩减指数。
// 降幂公式：a^b mod m = a^(b mod φ(m)+φ(m)) mod m（b≥φ(m)）；若 b<φ(m)，直接用 b。
// 这样写不要求 gcd(a,m)=1；只有互质时才能无条件直接把指数对 φ(m) 取模。
#include<bits/stdc++.h>
using namespace std;
#define int long long

int phi(int n){
    // 试除分解 m 的不同质因子，计算欧拉函数 φ(m)。
    int ans=n;
    for(int i=2;i*i<=n;i++){
        if(n%i==0){
            ans=ans/i*(i-1);
            while(n%i==0){
                n/=i;
            }
        }
    }
    if(n>1){
        ans=ans/n*(n-1);
    }
    return ans;
}

int power(int a,int b,int p){
    int ans=1;
    while(b){
        if(b&1){
            ans=(ans*a)%p;
        }
        b>>=1;
        a=(a*a)%p;
    }
    return ans;
}

signed main()
{
    int a,m;
    string b;
    cin>>a>>m>>b;

    if(m==1){
        cout<<0<<endl;
        return 0;
    }

    int p=phi(m);
    int val=0;
    int num=0;
    bool big=false;

    for(char c:b){
        // val 记录 b mod φ(m)；big 判断 b 是否已达到 φ(m)。
        int x=c-'0';
        val=(val*10+x)%p;
        if(!big){
            num=num*10+x;
            if(num>=p){
                big=true;
            }
```

```
        }
    }

    if(big){
        val+=p;
    }
    else{
        val=num;
    }
    cout<<power(a,val,m)<<endl;
    return 0;
}
```

```cpp
// 用途：求一般模数下 a^x≡b 的最小非负 x。
// 时间：期望 O(√p)。
// 空间：O(√p)。
// 注意：允许 gcd(a,p)>1；输入 0 0 0 结束。
// 例题：https://www.luogu.com.cn/problem/P4195
// 题意：模数可不与底数互质时，求最小离散对数。
// 思路：先逐步约去 gcd 并累计指数，再在互质部分运行 BSGS。
#include<bits/stdc++.h>
using namespace std;
#define int long long

int x,y,d,px,py;

void exgcd(int a,int b){
    if(b==0){
        d=a;
        x=1;
        y=0;
    }
    else{
        exgcd(b,a%b);
        px=x;
        py=y;
        x=py;
        y=px-py*(a/b);
    }
}

int inv(int a,int mod){
    exgcd(a,mod);
    return (x%mod+mod)%mod;
}

int gcd(int a,int b){
    return b==0?a:gcd(b,a%b);
}

int bsgs(int a,int b,int p){
    a%=p;
    b%=p;
    if(b==1){
        return 0;
    }

    int t=ceil(sqrt((long double)p));

    unordered_map<int,int>mp;
    mp.reserve(t*2);
    mp.max_load_factor(0.7);

    int now=b;
    for(int B=0;B<t;B++){
        mp[now]=B;
        now=(__int128)now*a%p;
    }

    int at=1;
    for(int i=1;i<=t;i++){
        at=(__int128)at*a%p;
    }

    now=1;
    for(int A=0;A<=t;A++){
        auto it=mp.find(now);
        if(it!=mp.end()){
            int ans=A*t-it->second;
```

```
            if(ans>=0){
                return ans;
            }
        }
        now=(__int128)now*at%p;
    }

    return -1;
}

// 先消去 a 与模数的公因子，再调用普通 BSGS。
int exbsgs(int a,int b,int mod){
    if(mod==1){
        return 0;
    }

    a%=mod;
    b%=mod;
    if(b==1){
        return 0;
    }

    int cnt=0;
    int k=1;

    while((d=gcd(a,mod))!=1){
        if(b%d!=0){
            return -1;
        }

        mod/=d;
        b/=d;
        k=(__int128)k*(a/d)%mod;
        cnt++;
        if(k==b){
            return cnt;
        }
    }

    b=(__int128)b*inv(k,mod)%mod;
    int ans=bsgs(a,b,mod);

    if(ans==-1){
        return -1;
    }

    return ans+cnt;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int a,b,p;
    while(cin>>a>>p>>b){
        if(a==0&&b==0&&p==0){
            break;
        }

        int ans=exbsgs(a,b,p);

        if(ans==-1){
            cout<<"No Solution\n";
        }
        else{
```

```
            cout<<ans<<"\n";
        }
    }
    return 0;
}
```

```cpp
// 用途：高斯消元求实数线性方程组。
// 时间：O(n□)。
// 空间：O(n□)。
// 例题：https://www.luogu.com.cn/problem/P3389
// 题意：求实数线性方程组的解，判断无解或多解。
// 思路：逐列选非零主元、消去其他行，再回代得到变量。
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 101;
// 0.0000001 == 1e-7
// 因为double类型有精度问题，所以认为
// 如果一个数字绝对值  <  sml，则认为该数字是0
// 如果一个数字绝对值  >= sml，则认为该数字不是0
const double sml = 1e-7;

int n;
double mat[MAXN][MAXN + 1];

void swap(int a, int b) {
    //swap的作用是用于交换两个数组的某一行
    //使用系统的内置函数可以  但是时间稍微差一点
    //在程序中 建议全部手写
    //但是手写的时候一定要注意中间数组tmp的类型是int 还是 double
    //并且手写的时候一定要注意列的范围  不能缺少
    double tmp[MAXN + 1];
    for (int j = 0; j <= n+1; j++) {
        tmp[j] = mat[a][j];
        mat[a][j] = mat[b][j];
        mat[b][j] = tmp[j];
    }
}

//这个函数的作用是求出mat矩阵的解  n是x1 x2    xn
//如果无解返回0  有解返回1
int gauss(int n) {
    for (int i = 1; i <= n; i++) {
        //每次求解一个xi  也不是最终的值
        //只是将其他式子中的xi的系数消除为0
        //只保留自己的系数  最终将系数化为1
        int max = i;
        for (int j = 1; j <= n; j++) {
            if (j < i && abs(mat[j][j]) >= sml) {
                continue;
            }
            if (abs(mat[j][i]) > abs(mat[max][i])) {
                max = j;
            }
        }
        swap(i, max);//交换改行最大值
        if (abs(mat[i][i]) < sml) {
            //表示xi为0  那么不可能有解了
            return 0;
        }
        if (abs(mat[i][i]) >= sml) {
            //消去其他式子中的xi
            double tmp = mat[i][i];
            for (int j = i; j <= n + 1; j++) {
                mat[i][j] /= tmp;
            }
            for (int j = 1; j <= n; j++) {
                if (i != j) {
                    double rate = mat[j][i] / mat[i][i];
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] -= mat[i][k] * rate;
                    }
```

```
                }
            }
        }
    }
    return 1;
}

int main()
{
    cin>>n;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n+1;j++){
            cin>>mat[i][j];
        }
    }
    if(gauss(n)==1){
        for(int i=1;i<=n;i++){
            printf("%.2lf\n",mat[i][n+1]);
        }
    }
    else{
        cout<<"No Solution"<<endl;
    }
    return 0;
}
```

```cpp
// 用途：求无向带权图从 1 到 n 的路径异或和最大值。
// 时间：O((n+m)B)，B=61 为权值二进制位数。
// 空间：O(n+m+B)。
// 注意：图保证连通，可有重边和自环；边权最高 10^18，须用 long long。
// 例题：https://www.luogu.com.cn/problem/P4151
// 题意：求图上从 1 到 n 的路径边权异或和最大值。
// 思路：DFS 得一条基础路径，把每条环的异或值插入线性基后贪心增大答案。
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 50001;
const int MAXM = 200002;
const int BIT = 60;

// 链式前向星
int head[MAXN];
int Next[MAXM];
int to[MAXM];
long long weight[MAXM];
int cnt=2; // 无向边成对放在 e、e^1；0 表示空边。

// 所有环的异或和构建的线性基
long long basis[BIT + 1];
// 某个节点在dfs过程中是否被访问过
bool visited[MAXN];
// 从头结点到当前节点的异或和
long long path[MAXN];
int stk[MAXN];

void addEdge(int u, int v, long long w) {
    Next[cnt] = head[u];
    to[cnt] = v;
    weight[cnt] = w;
    head[u] = cnt++;
}

bool insert(long long num) {
    for (int i = BIT; i >= 0; i--)
        if ((num >> i) & 1LL) {
            if (basis[i] == 0) {
                basis[i] = num;
                return true;
            }
            num ^= basis[i];
        }
    return false;
}

void dfs(int start) {
    // 任取一棵生成树记录 1 到各点的异或值；深链用定长栈避免爆栈。
    int top = 0;
    visited[start] = true;
    stk[++top] = start;
    while (top) {
        int u = stk[top--];
        for (int e = head[u]; e; e = Next[e]) {
            int v = to[e];
            if (!visited[v]) {
                visited[v] = true;
                path[v] = path[u] ^ weight[e];
                stk[++top] = v;
            } else {
                // 非树边形成的环异或值；反向树边只产生 0。
                insert(path[u] ^ path[v] ^ weight[e]);
            }
        }
    }
```

```
}

long long query(long long init) {
    // 每个环可以让路径多绕一次，线性基贪心提高基础路径异或值。
    for (int i = BIT; i >= 0; i--) {
        init = max(init, init ^ basis[i]);
    }
    return init;
}

int main()
{
    int n ;
    int m ;
    cin>>n>>m;
    for (int i = 1; i <= m; i++) {
        int u ;
        int v ;
        long long w ;
        cin>>u>>v>>w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    dfs(1);
    cout<<query(path[n])<<endl;
    return 0;
}
```

```cpp
// https://www.luogu.com.cn/problem/P3455
// P3455 [POI2007] ZAP-Queries
// 题意：
// 给定 a,b,d，求有多少对 (x,y) 满足：
// 1 <= x <= a     1 <= y <= b    gcd(x,y) = d
// 先同时除以 d：
// gcd(x,y)=d  等价于  gcd(x/d,y/d)=1
// 所以问题转化为：
// 求 1<=x<=n, 1<=y<=m 中 gcd(x,y)=1 的数对数量
// 利用莫比乌斯函数：
// [gcd(x,y)=1]  = sum_{k|x且k|y} mu(k)
//
// 交换枚举顺序：
// ans = sum_{k=1}^{min(n,m)}  mu(k) * floor(n/k) * floor(m/k)
// 由于 floor(n/k)、floor(m/k) 都具有整除分块性质
// 对于一段 [l,r]，它们的值保持不变
// 所以维护 mu 的前缀和：
// sumMu[i] = mu[1] + ... + mu[i]
// 每一段贡献：  (sumMu[r]-sumMu[l-1]) * (n/l) * (m/l)
// 时间复杂度： 预处理 O(N)
// 单次询问约 O(sqrt(N))
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 5e4+5;

int prime[MAXN];
bool vis[MAXN];
int cnt;
// 莫比乌斯函数
int mu[MAXN];
int sum[MAXN];

void prepare(int n){
    mu[1]=sum[1]=1;
    for(int i=2;i<=n;i++){
        if(!vis[i]){
            prime[++cnt]=i;
            mu[i]=-1;
        }
        for(int j=1;j<=cnt&&(i*prime[j]<=n);j++){
            vis[i*prime[j]]=true;
            if(i%prime[j]==0){
                mu[i*prime[j]]=0;
                break;
            }
            else{
                mu[i*prime[j]]=-mu[i];
            }
        }
        sum[i]=sum[i-1]+mu[i];
    }
}

int calc(int n,int m){
    long long ans=0;
    if(n>m){
        swap(n,m);
    }
    for(int l=1,r;l<=n;l=r+1){
        r=min(n/(n/l),m/(m/l));
        ans+=(sum[r]-sum[l-1])*(m/l)*(n/l);
    }
    return ans;
}

```

```
signed main()
{
    prepare(50000);
    int T;
    cin>>T;
    while(T--){
        int a,b,d;
        cin>>a>>b>>d;
        cout<<calc(a/d,b/d)<<endl;
    }
    return 0;
}
```

```cpp
// https://www.luogu.com.cn/problem/P4245
// 这道题给出的模数不是NTT的合适模数
// 所以我们采用多个模数结合中国剩余定理完成
// 中国剩余定理只能求的是一个值对三个模数的lcm的取模后的结果
// 但是这道题的正确性的保证是可以通过三个模数的lcm很大保证是对的
#include<bits/stdc++.h>
using namespace std;
#define int long long
typedef __int128 i128;
const int MAXN = 1e5+5;
const int MAXT = 4e5+5;
const int P[3]={998244353,1004535809,469762049};
const int g=3;
const int gi[3]={332748118,334845270,156587350};

int n,m,len,mod;
int A[MAXT];
int B[MAXT];
int a[MAXN];
int b[MAXN];
int r[3][MAXT];
int ans[MAXT];
i128 val[MAXT];

int qpow(int a,int b,int P){
    int ans=1;
    while(b){
        if(b&1){
            ans=ans*a%P;
        }
        a=a*a%P;
        b>>=1;
    }
    return ans;
}

// 递归 NTT
void NTT(int A[],int n,int op,int P,int g,int gi){
    if(n==1){
        return;
    }

    int A1[n/2],A2[n/2];
    // 拆分奇偶项
    for(int i=0;i<n/2;i++){
        A1[i]=A[i*2];
        A2[i]=A[i*2+1];
    }

    // 递归分治
    NTT(A1,n/2,op,P,g,gi);
    NTT(A2,n/2,op,P,g,gi);

    // 单位根
    int g1=qpow(op==1?g:gi,(P-1)/n,P);
    int gk=1;

    // 蝶形运算
    for(int i=0;i<n/2;i++){
        A[i]=(A1[i]+A2[i]*gk)%P;
        A[i+n/2]=(A1[i]-A2[i]*gk%P+P)%P;
        gk=gk*g1%P;
    }
}

i128 d, x, y, px, py;
```

```
void exgcd(i128 a, i128 b) {
    if (b == 0) {
        d = a;
        x = 1;
        y = 0;
    } else {
        exgcd(b, a % b);
        px = x;
        py = y;
        x = py;
        y = px - py * (a / b);
    }
}

void crt(){
    i128 lcm=1;
    for (int i = 0; i < 3; i++) {
        lcm = lcm * P[i];
    }
    i128 ai;
    for(int op=0;op<3;op++){
        ai=lcm/P[op];
        exgcd(ai,P[op]);
        int t=(x%P[op]+P[op])%P[op];
        for(int i=0;i<=n+m;i++){
            i128 ci=(r[op][i]*ai)%lcm*t%lcm;
            val[i]=(val[i]+ci)%lcm;
        }
    }

    for(int i=0;i<=n+m;i++){
        ans[i]=val[i]%mod;
    }
}

signed main()
{
    cin>>n>>m>>mod;
    for(int i=0;i<=n;i++){
        cin>>a[i];
    }
    for(int i=0;i<=m;i++){
        cin>>b[i];
    }

    len=1;
    while(len<(n+m+1)){
        len<<=1;
    }

    for(int op=0;op<3;op++){
        for(int i=0;i<len;i++){
            A[i]=B[i]=0;
        }

        for(int i=0;i<=n;i++){
            A[i]=a[i]%P[op];
        }
        for(int i=0;i<=m;i++){
            B[i]=b[i]%P[op];
        }

        NTT(A,len,1,P[op],g,gi[op]);
        NTT(B,len,1,P[op],g,gi[op]);
        for(int i=0;i<len;i++){
            A[i]=A[i]*B[i]%P[op];
```

```
        }
        NTT(A,len,-1,P[op],g,gi[op]);
        int invlen=qpow(len,P[op]-2,P[op]);
        for(int i=0;i<len;i++){
            r[op][i]=A[i]*invlen%P[op];
        }
    }

    crt();

    for(int i=0;i<=n+m;i++){
        cout<<(ans[i]%mod)<<' ';
    }
    return 0;
}
```

```cpp
// https://www.luogu.com.cn/problem/P4238
// 题意：给定多项式 A(x)，求 B(x)，满足 A(x)*B(x) ≡ 1 (mod x^n)
// 所有系数对 998244353 取模
// 核心公式：  B_new = B * (2 - A * B) (mod x^n)
// 时间复杂度：O(n log n)
// 一个多项式可以求逆的条件是他的常数项（x^0的系数在当前模数下有逆元）
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 2e5+5;
const int MAXT = 4e5+5;
const int P = 998244353;
const int g = 3;
const int gi = 332748118;

int n;
int a[MAXN];
int b[MAXN];
int A[MAXT];
int B[MAXT];

// 快速幂
int qpow(int a,int b){
    int ans=1;
    while(b){
        if(b&1){
            ans=ans*a%P;
        }
        a=a*a%P;
        b>>=1;
    }
    return ans;
}

// 递归 NTT
void NTT(int A[],int n,int op){
    if(n==1){
        return;
    }

    int A1[n/2],A2[n/2];
    // 拆分奇偶项
    for(int i=0;i<n/2;i++){
        A1[i]=A[i*2];
        A2[i]=A[i*2+1];
    }

    // 递归分治
    NTT(A1,n/2,op);
    NTT(A2,n/2,op);

    // 单位根
    int g1=qpow(op==1?g:gi,(P-1)/n);
    int gk=1;

    // 蝶形运算
    for(int i=0;i<n/2;i++){
        A[i]=(A1[i]+A2[i]*gk)%P;
        A[i+n/2]=(A1[i]-A2[i]*gk%P+P)%P;
        gk=gk*g1%P;
    }
}

// 多项式求逆
// 求 B(x)=A(x)^(-1) mod x^n
void poly_inv(int n){
```

```
    if(n==1){
        b[0]=qpow(a[0],P-2);
        return;
    }

    poly_inv(n>>1);
    int len=n<<1;

    // 构造多项式
    // A 为 a 的前 n 项
    // B 为已经求出的 b 的前 n/2 项
    for(int i=0;i<len;i++){
        A[i]=(i<n?a[i]:0);
        B[i]=(i<n/2?b[i]:0);
    }

    NTT(A,len,1);
    NTT(B,len,1);
    for(int i=0;i<len;i++){
        int t=(2-A[i]*B[i]%P+P)%P;
        A[i]=B[i]*t%P;
    }
    NTT(A,len,-1);

    int invn=qpow(len,P-2);
    for(int i=0;i<n;i++){
        b[i]=A[i]*invn%P;
    }
}

signed main()
{
    cin>>n;
    for(int i=0;i<n;i++){
        cin>>a[i];
    }

    int m=1;
    while(m<n){
        m<<=1;
    }
    poly_inv(m);

    for(int i=0;i<n;i++){
        cout<<b[i]<<' ';
    }
    return 0;
}
```

```cpp
// 用途：康托展开求排列的字典序排名。
// 时间：O(n log n)。
// 空间：O(n)。
// 例题：https://www.luogu.com.cn/problem/P5367
// 题意：求给定排列在所有排列中字典序排第几。
// 思路：逐位统计未使用且更小的数，乘剩余位置阶乘求贡献。
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 1000001;
const int MOD = 998244353;

int arr[MAXN];
// 阶乘余数表
int fac[MAXN];
// 树状数组
int tree[MAXN];
int n;

//以下部分是树状数组用于求和
int lowbit(int x){
    return x & -x;
}

void add(int i,int v){
    while(i<=n){
        tree[i]+=v;
        i+=lowbit(i);
        //每一次都加上自己最右侧的1
        //相当于把所有包含该状态的数字都增加
    }
}

//这个函数求的是从1到i所有数字的和
int sum(int i){
    int ans=0;
    while(i>0){
        ans+=tree[i];
        i-=lowbit(i);
        //每一次都减去自己最右侧的1
        //相当于将整个区间拆分开为几个小区间加速  和乘法快速幂相似
    }
    return ans;
}

int query(int l,int r){
    return sum(r)-sum(l-1);
}

signed main()
{
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>arr[i];
    }

    //建立阶乘表
    fac[0]=1;
    for(int i=1;i<n;i++){
        fac[i]=(fac[i-1]*i)%MOD;
    }

    for(int i=1;i<=n;i++){
        add(i,1);
    }
    //将树状数组初始化
```

```
    long long ans=0;
    for(int i=1;i<=n;i++){
        ans=(ans+(fac[n-i]*sum(arr[i]-1))%MOD)%MOD;
        add(arr[i],-1);//将这一位的信息去除
    }
    ans=(ans+1)%MOD;
    cout<<ans<<endl;
    return 0;
}
```

```cpp
// 用途：计算分糖豆博弈的 SG 值，统计能使对手必败的首步。
// 时间：O(n□)。
// 空间：O(n)。
// 例题：https://www.luogu.com.cn/problem/P3185
// 题意：给定糖豆分堆规则，求能让对手必败的首步数量。
// 思路：递归记忆化各状态 SG 值，异或判断转移后是否为 0。
#include <iostream>
#include <cstring>
#include <vector>
#include <algorithm>
using namespace std;

const int MAXN = 21; // 最大瓶子数量
const int MAXV = 101; // SG 值的最大范围

int nums[MAXN]; // 每个瓶子中的糖豆数量
int sg[MAXN]; // 每个位置的 SG 值
bool appear[MAXV]; // 用于计算 SG 值的辅助数组

// 预处理 SG 值
void build() {
    for (int i = 1; i < MAXN; i++) {
        memset(appear, false, sizeof(appear)); // 重置 appear 数组
        for (int j = i - 1; j >= 0; j--) {
            for (int k = j; k >= 0; k--) {
                appear[sg[j] ^ sg[k]] = true; // 标记 SG 值的组合
            }
        }
        for (int s = 0; s < MAXV; s++) {
            if (!appear[s]) {
                sg[i] = s; // 找到最小的未出现的 SG 值
                break;
            }
        }
    }
}

// 计算当前状态的解
string compute(int n) {
    int eor = 0; // 所有糖果的 SG 值异或和
    for (int i = n - 1; i >= 0; i--) {
        if (nums[i] % 2 == 1) {
            eor ^= sg[i]; // 奇数个糖豆才参与异或
        }
    }
    if (eor == 0) {
        return "-1 -1 -1\n0"; // 无法获胜
    }
    int cnt = 0, a = -1, b = -1, c = -1, pos;
    for (int i = n - 1; i >= 1; i--) {
        if (nums[i] > 0) {
            for (int j = i - 1; j >= 0; j--) {
                for (int k = j; k >= 0; k--) {
                    pos = eor ^ sg[i] ^ sg[j] ^ sg[k]; // 计算新的 SG 值
                    if (pos == 0) {
                        cnt++; // 统计可行解的数量
                        if (a == -1) {
                            a = i;
                            b = j;
                            c = k; // 记录字典序最小的解
                        }
                    }
                }
            }
        }
    }
```

```
    return to_string(n - 1 - a) + " " + to_string(n - 1 - b) + " " + to_string(n - 1 - c) + "\n"
     + to_string(cnt);
}

int main() {
    build(); // 预处理 SG 值
    int t, n;
    cin >> t; // 测试用例数量
    while (t--) {
        cin >> n; // 瓶子数量
        for (int j = n - 1; j >= 0; j--) {
            cin >> nums[j]; // 输入每个瓶子中的糖豆数量
        }
        cout << compute(n) << endl; // 输出结果
    }
    return 0;
}
```

```cpp
// 用途：在 n 对数中删去 k 对，使剩余数的 Σx/Σy 最大。
// 时间：O(n log n log(R/ε))，每轮排序选出最大的 k 个结余。
// 空间：O(n)。
// 注意：分母需为正；二分取上界作百分数四舍五入，n<=1000。
// 例题：https://www.luogu.com.cn/problem/P10505
// 题意：删去 k 对数，使剩余数对的总分子除总分母最大。
// 思路：二分答案 x，将每对变为 a-xb，选最大的 n-k 项判断和是否非负。
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 1001;

struct node {
    double x,y,r;
}arr[MAXN];
int n,k;

bool cmp(node a,node b){
    return a.r>b.r;
}

bool check(double x) {
    // x固定的情况下，得到所有数据的结余
    for (int i = 1; i <= n; i++) {
        arr[i].r = arr[i].x - x * arr[i].y;
    }
    // 结余从大到小排序
    sort(arr + 1, arr + n + 1, cmp);
    // 最大k个结余的累加和
    double sum = 0;
    for (int i = 1; i <= k; i++) {
        sum += arr[i].r;
    }
    return sum >= 0;
}

int main()
{
    cin>>n>>k;
    while((n+k)!=0){
        k = n - k;
        for (int i = 1; i <= n; i++) {
            cin>>arr[i].x;
        }
        for (int i = 1; i <= n; i++) {
            cin>>arr[i].y;
        }
        double l = 0, r = 0, x;
        for (int i = 1; i <= n; i++) {
            r += arr[i].x;
        }
        // 固定迭代次数，保持 l 可行、r 不可行的二分边界。
        for (int iter = 0; iter < 70; ++iter) {
            x = (l + r) / 2;
            if (check(x)) {
                l = x;
            } else {
                r = x;
            }
        }
        cout<<llround(r * 100.0)<<endl;
        cin>>n>>k;
    }
    return 0;
}
```

```
// 用途：回答多次询问：树上是否存在距离恰为 k 的两个不同节点。
// 时间：O(nm log n)，n 为点数，m 为询问数。
// 空间：O(n+K)，K 为最大询问距离，用布尔数组记录已处理的距离。
// 注意：边权为正，n≤10000、m≤100、k≤10000000；距离超过最大询问可剪枝。
// 例题：https://www.luogu.com.cn/problem/P3806
// 题意：给定带权树和多个 k，逐个判断有没有一对点的路径长度等于 k。
// 思路：每层选重心；先查当前子树距离能否与先前子树凑成 k，再插入当前子树。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 10001;
const int MAXM = 101;
const int MAXV = 10000001;

int n, m;

int query[MAXM];
int maxk;

int head[MAXN];
int nxt[MAXN << 1];
int to[MAXN << 1];
int weight[MAXN << 1];
int cntg;

bool vis[MAXN];
int siz[MAXN];

// 从u出发，到当前子树的每个节点，收集路径权值和
int cur[MAXN];
int cntc;

// 从u出发，到之前所有子树的每个节点，收集路径权值和
int all[MAXN];
int cnta;

// 使用数组替代哈希表，因为哈希表常数时间大，会超时
bool exist[MAXV];

bool ans[MAXM];

void addEdge(int u, int v, int w) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    weight[cntg] = w;
    head[u] = cntg;
}

void getSize(int u, int fa) {
    siz[u] = 1;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (v != fa && !vis[v]) {
            getSize(v, u);
            siz[u] += siz[v];
        }
    }
}

int getCentroid(int u, int fa) {
    getSize(u, fa);
    int half = siz[u] >> 1;
    bool find = false;
    while (!find) {
        find = true;
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
```

```
            if (v != fa && !vis[v] && siz[v] > half) {
                fa = u;
                u = v;
                find = false;
                break;
            }
        }
    }
    return u;
}

void dfs(int u, int fa, int dis) {
    if (dis > maxk) {
        return;
    }
    cur[++cntc] = dis;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (v != fa && !vis[v]) {
            dfs(v, u, dis + weight[e]);
        }
    }
}

void calc(int u) {
    cnta = 0;
    exist[0] = true; // 重心本身到重心的距离为 0。
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (!vis[v]) {
            cntc = 0;
            dfs(v, u, weight[e]);
            // 先查询、后加入：只会配到前面子树的点，避免在同一子树内配对。
            for (int i = 1; i <= m; i++) {
                for (int j = 1; !ans[i] && j <= cntc; j++) {
                    if (query[i] - cur[j] >= 0) {
                        ans[i] |= exist[query[i] - cur[j]];
                    }
                }
            }
            for (int i = 1; i <= cntc; i++) {
                all[++cnta] = cur[i];
                exist[cur[i]] = true;
            }
        }
    }
    // 只清除这一层写过的距离；下一个重心不能沿用旧记录。
    for (int i = 1; i <= cnta; i++) {
        exist[all[i]] = false;
    }
}

void solve(int u) {
    vis[u] = true;
    calc(u);
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (!vis[v]) {
            solve(getCentroid(v, u));
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    cin >> n >> m;
    for (int i = 1, u, v, w; i < n; i++) {
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    for (int i = 1; i <= m; i++) {
        cin >> query[i];
        maxk = max(maxk, query[i]);
    }
    solve(getCentroid(1, 0));
    for (int i = 1; i <= m; i++) {
        if (ans[i]) {
            cout << "AYE" << '\n';
        } else {
            cout << "NAY" << '\n';
        }
    }
    return 0;
}
```

```cpp
// 用途：统计带权树上距离不超过 k 的无序点对，不把 (x,x) 算作点对。
// 时间：O(n log□ n)，每层点分治都要排序后用双指针计数。
// 空间：O(n)。
// 注意：边权非负，才能剪掉到重心距离已经超过 k 的分支。
// 例题：https://www.luogu.com.cn/problem/P4178
// 题意：给定一棵树和 k，求距离≤k 的不同节点对数量。
// 思路：先算当前连通块所有点对，再减去每个子树内部的点对，留下路径经过重心的贡献。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 50001;

int n, k;

int head[MAXN];
int nxt[MAXN << 1];
int to[MAXN << 1];
int weight[MAXN << 1];
int cntg;

bool vis[MAXN];
int siz[MAXN];
int order[MAXN], parentNode[MAXN];
int stackNode[MAXN], stackParent[MAXN], stackDist[MAXN];

int disArr[MAXN];
int cnta;

void addEdge(int u, int v, int w) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    weight[cntg] = w;
    head[u] = cntg;
}

void getSize(int u, int fa) {
    // 连通块可能是一条 4 万点长链，显式记录 DFS 顺序以免递归栈溢出。
    int count = 1;
    order[0] = u;
    parentNode[u] = fa;
    for (int i = 0; i < count; ++i) {
        int x = order[i];
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != parentNode[x] && !vis[v]) {
                parentNode[v] = x;
                order[count++] = v;
            }
        }
    }
    for (int i = count - 1; i >= 0; --i) {
        int x = order[i];
        siz[x] = 1;
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != parentNode[x] && !vis[v]) siz[x] += siz[v];
        }
    }
}

int getCentroid(int u, int fa) {
    getSize(u, fa);
    int half = siz[u] >> 1;
    bool find = false;
    while (!find) {
        find = true;
        for (int e = head[u]; e; e = nxt[e]) {
```

```
            int v = to[e];
            if (v != fa && !vis[v] && siz[v] > half) {
                fa = u;
                u = v;
                find = false;
                break;
            }
        }
    }
    return u;
}

void dfs(int u, int fa, int dis) {
    int top = 0;
    stackNode[top] = u;
    stackParent[top] = fa;
    stackDist[top++] = dis;
    while (top) {
        --top;
        int x = stackNode[top], p = stackParent[top], d = stackDist[top];
        if (d > k) continue;
        disArr[++cnta] = d;
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != p && !vis[v]) {
                stackNode[top] = v;
                stackParent[top] = x;
                stackDist[top++] = d + weight[e];
            }
        }
    }
}

long long calc(int u, int dis) {
    cnta = 0;
    dfs(u, 0, dis);
    long long ans = 0;
    sort(disArr + 1, disArr + cnta + 1);
    // 双指针统计距离和≤k 的无序点对；不包括同一点。
    for (int l = 1, r = cnta; l <= r; ) {
        if (disArr[l] + disArr[r] <= k) {
            ans += r - l;
            l++;
        } else {
            r--;
        }
    }
    return ans;
}

long long solve(int u) {
    vis[u] = true;
    long long ans = calc(u, 0); // 先计当前连通块的全部点对。
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (!vis[v]) {
            ans -= calc(v, weight[e]); // 同一子树内的路径不经过重心，先扣掉。
            ans += solve(getCentroid(v, u));
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    cin >> n;
    for (int i = 1, u, v, w; i < n; i++) {
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    cin >> k;
    cout << solve(getCentroid(1, 0)) << '\n';
    return 0;
}
```

```cpp
// 用途：维护树上点权修改，并查询距指定节点不超过 k 的点权和。
// 时间：O((n+q)log□ n)。
// 空间：O(n log n)。
// 例题：https://www.luogu.com.cn/problem/P6329
// 题意：改单点权，询问距离某点不超过 k 的点权和。
// 思路：点分树祖先维护按距离的统计，查询时加总并扣除重复部分。
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100001;
const int MAXT = 10000001;

int n, m;
int arr[MAXN];

int head[MAXN];
int nxt[MAXN << 1];
int to[MAXN << 1];
int cntg;

// 这里使用重链剖分
int fa[MAXN];
int dep[MAXN];
int siz[MAXN];
int son[MAXN];
int top[MAXN];
int workOrder[MAXN], workParent[MAXN], workCount;

bool vis[MAXN];
// 只需要记录自己的点分树上的父亲节点即可
// sz数组是和树链剖分公用的
int centfa[MAXN];

// 正负树  使用开点线段树实现
// 每个节点的正树上记录着自己的点分树上的子树上的点到自己距离为i的权值之和
// 每个节点的负树上记录着自己这个点分树上的子树上的所有节点在自己的父亲节点中的贡献
// 这样记录数据可以方便避免重复计算某个节点的子树贡献
int addTree[MAXN];
int minusTree[MAXN];
int ls[MAXT];
int rs[MAXT];
int sum[MAXT];
int cntt;

void addEdge(int u, int v) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    head[u] = cntg;
}

void dfs1(int u, int f) {
    // 原树可能是 10 万点长链，先求父亲在前的顺序，再倒序统计子树大小。
    workCount = 1;
    workOrder[0] = u;
    fa[u] = f;
    dep[u] = dep[f] + 1;
    for (int i = 0; i < workCount; ++i) {
        int x = workOrder[i];
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != fa[x]) {
                fa[v] = x;
                dep[v] = dep[x] + 1;
                workOrder[workCount++] = v;
            }
        }
    }
```

```
    for (int i = workCount - 1; i >= 0; --i) {
        int x = workOrder[i];
        siz[x] = 1;
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != fa[x]) {
                siz[x] += siz[v];
                if (son[x] == 0 || siz[son[x]] < siz[v]) son[x] = v;
            }
        }
    }
}

void dfs2(int u, int t) {
    top[u] = t;
    // workOrder 中父节点总在子节点之前，可以直接继承重链链顶。
    for (int i = 1; i < workCount; ++i) {
        int v = workOrder[i], p = fa[v];
        top[v] = (v == son[p] ? top[p] : v);
    }
}

int getLca(int a, int b) {
    while (top[a] != top[b]) {
        if (dep[top[a]] <= dep[top[b]]) {
            b = fa[top[b]];
        } else {
            a = fa[top[a]];
        }
    }
    return dep[a] <= dep[b] ? a : b;
}

int getDist(int x, int y) {
    return dep[x] + dep[y] - (dep[getLca(x, y)] << 1);
}

void getSize(int u, int fa) {
    int count = 1;
    workOrder[0] = u;
    workParent[u] = fa;
    for (int i = 0; i < count; ++i) {
        int x = workOrder[i];
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != workParent[x] && !vis[v]) {
                workParent[v] = x;
                workOrder[count++] = v;
            }
        }
    }
    for (int i = count - 1; i >= 0; --i) {
        int x = workOrder[i];
        siz[x] = 1;
        for (int e = head[x]; e; e = nxt[e]) {
            int v = to[e];
            if (v != workParent[x] && !vis[v]) siz[x] += siz[v];
        }
    }
}

int getCentroid(int u, int fa) {
    getSize(u, fa);
    int half = siz[u] >> 1;
    bool find = false;
    while (!find) {
```

```
        find = true;
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (v != fa && !vis[v] && siz[v] > half) {
                fa = u;
                u = v;
                find = false;
                break;
            }
        }
    }
    return u;
}

void centroidTree(int u, int fa) {
    // 记录自己在点分树上的父亲节点
    centfa[u] = fa;
    vis[u] = true;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (!vis[v]) {
            centroidTree(getCentroid(v, u), u);
        }
    }
}

int add(int jobi, int jobv, int l, int r, int i) {
    if (i == 0) {
        i = ++cntt;
    }
    if (l == r) {
        sum[i] += jobv;
    } else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            ls[i] = add(jobi, jobv, l, mid, ls[i]);
        } else {
            rs[i] = add(jobi, jobv, mid + 1, r, rs[i]);
        }
        sum[i] = sum[ls[i]] + sum[rs[i]];
    }
    return i;
}

int query(int jobl, int jobr, int l, int r, int i) {
    if (i == 0) {
        return 0;
    }
    if (jobl <= l && r <= jobr) {
        return sum[i];
    }
    int mid = (l + r) >> 1;
    int ans = 0;
    if (jobl <= mid) {
        ans += query(jobl, jobr, l, mid, ls[i]);
    }
    if (jobr > mid) {
        ans += query(jobl, jobr, mid + 1, r, rs[i]);
    }
    return ans;
}

void add(int x, int v) {
    addTree[x] = add(0, v, 0, n - 1, addTree[x]);
    for (int cur = x, fa = centfa[cur]; fa > 0; cur = fa, fa = centfa[cur]) {
        int dist = getDist(x, fa);
```

```
        // 在父亲的正树上加  在自己的负树上减
        addTree[fa] = add(dist, v, 0, n - 1, addTree[fa]);
        minusTree[cur] = add(dist, v, 0, n - 1, minusTree[cur]);
    }
}

int query(int x, int k) {
    int ans = query(0, k, 0, n - 1, addTree[x]);
    for (int cur = x, fa = centfa[cur]; fa > 0; cur = fa, fa = centfa[cur]) {
        int dist = getDist(x, fa);
        if (k - dist >= 0) {
            // 查询父亲正树上的信息 同时减去自己的重复贡献
            ans += query(0, k - dist, 0, n - 1, addTree[fa]);
            ans -= query(0, k - dist, 0, n - 1, minusTree[cur]);
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs1(1, 0);
    dfs2(1, 1);
    centroidTree(getCentroid(1, 0), 0);
    for (int i = 1; i <= n; i++) {
        // 每个节点的点权都是调用add函数实现
        add(i, arr[i]);
    }
    int lastAns = 0;
    for (int i = 1, op, x, y; i <= m; i++) {
        cin >> op >> x >> y;
        x ^= lastAns;
        y ^= lastAns;
        if (op == 0) {
            lastAns = query(x, y);
            cout << lastAns << '\n';
        } else {
            // 点权变化也是调用add实现
            add(x, y - arr[x]);
            arr[x] = y;
        }
    }
    return 0;
}
```

```cpp
// 用途：离线处理边的添加、删除和两点是否连通的查询。
// 时间：O((n+q)log q log n)。
// 空间：O(n□+q log q)，邻接端点的上次出现时间用 n×n 数组记录。
// 注意：可撤销并查集不能路径压缩；任务只存加边操作编号，避免复制端点。
// 例题：https://loj.ac/p/121
// 题意：边可增删，多次询问两点当前是否连通。
// 思路：边的生存期挂在线段树节点，DFS 时合并并查集，回溯时撤销。
#include <bits/stdc++.h>
using namespace std;
// 点的数量最大值
const int MAXN = 5001;
// 操作数量最大值
const int MAXM = 500001;
// 一条边的生存区间最多挂 O(log m) 个节点，总任务上界按 m log m 开。
const int MAXT = MAXM * 20;

int n, m;

// 操作类型op、端点u、端点v
int op[MAXM];
int u[MAXM];
int v[MAXM];

// last[x][y] : 点x和点y的边，上次出现的时间点
int last[MAXN][MAXN];

// 可撤销并查集
int father[MAXN];
int siz[MAXN];
int rollback[MAXN][2];
int opsize = 0;

// 线段树每个区间拥有哪些任务的列表，链式前向星表示
int head[MAXM << 2];
int nxt[MAXT];
int taskOp[MAXT]; // 原加边操作编号；端点从 u/v 数组读取。
int cnt = 0;

// ans[i]为第i条操作的答案，只有查询操作才有答案
bool ans[MAXM];

void addEdge(int i, int edgeOp) {
    nxt[++cnt] = head[i];
    taskOp[cnt] = edgeOp;
    head[i] = cnt;
}

//可撤销并查集  没有扁平化操作
int find(int i) {
    while (i != father[i]) {
        i = father[i];
    }
    return i;
}

// 必须要小挂大 不然时间复杂度会达到O(n)级别
void Union(int x, int y) {
    int fx = find(x);
    int fy = find(y);
    if (siz[fx] < siz[fy]) {
        int tmp = fx;
        fx = fy;
        fy = tmp;
    }
    father[fy] = fx;
    siz[fx] += siz[fy];
```

```
    rollback[++opsize][0] = fx;
    rollback[opsize][1] = fy;
}

void undo() {
    int fx = rollback[opsize][0];
    int fy = rollback[opsize--][1];
    father[fy] = fy;
    siz[fx] -= siz[fy];
}

// 把加边操作挂到边存在的时间区间 [jobl, jobr]。
void add(int jobl, int jobr, int edgeOp, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        addEdge(i, edgeOp);
    } else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            add(jobl, jobr, edgeOp, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, edgeOp, mid + 1, r, i << 1 | 1);
        }
    }
}

//dfs遍历树
void dfs(int l, int r, int i) {
    int unionCnt = 0;//记录这个节点上进行了多少次合并操作  方便最后的撤销操作
    for (int ei = head[i], x, y, fx, fy; ei > 0; ei = nxt[ei]) {
        x = u[taskOp[ei]];
        y = v[taskOp[ei]];
        fx = find(x);
        fy = find(y);
        if (fx != fy) {
            Union(fx, fy);
            unionCnt++;
        }
    }
    if (l == r) {
        //在每个叶节点统计答案
        if (op[l] == 2) {
            ans[l] = find(u[l]) == find(v[l]);
        }
    } else {
        int mid = (l + r) >> 1;
        dfs(l, mid, i << 1);
        dfs(mid + 1, r, i << 1 | 1);
    }
    for (int j = 1; j <= unionCnt; j++) {
        undo();
    }
}

//整理所有的信息
void prepare() {
    for (int i = 1; i <= n; i++) {
        father[i] = i;
        siz[i] = 1;
    }
    for (int i = 1, t, x, y; i <= m; i++) {
        t = op[i];
        x = u[i];
        y = v[i];
        if (t == 0) {
            //如果是连线
```

```
            last[x][y] = i;
        } else if (t == 1) {
            //如果是删除边  那么就是意味着在 last~i-1 这个时间段内 x和y存在边的关系
            add(last[x][y], i - 1, last[x][y], 1, m, 1);
            last[x][y] = 0;
        }
    }
    for (int x = 1; x <= n; x++) {
        for (int y = x + 1; y <= n; y++) {
            //表示直到最后还没有被删除的边  说明他们从最后一个加入开始一直生效到最后
            if (last[x][y] != 0) {
                add(last[x][y], m, last[x][y], 1, m, 1);
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1, t, x, y; i <= m; i++) {
        cin >> t >> x >> y;
        op[i] = t;
        u[i] = min(x, y);
        v[i] = max(x, y);
    }
    prepare();
    dfs(1, m, 1);
    for (int i = 1; i <= m; i++) {
        if (op[i] == 2) {
            if (ans[i]) {
                cout << "Y" << "\n";
            } else {
                cout << "N" << "\n";
            }
        }
    }
   return 0;
}
```

```cpp
// 用途：有向容量网络最大流；输入 n m s t 和 m 条 u v 容量。
// 时间：一般上界 O(n□m)。
// 空间：O(n+m)。
// 注意：n<=200，m<=5000；同点源汇输出 0。
// 例题：https://www.luogu.com.cn/problem/P3376
// 题意：求容量网络中源点到汇点能送出的最大流量。
// 思路：BFS 建分层图，DFS 沿允许边反复增广。
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 205;
const int MAXM = 10005;
const int INF = 1e10;

int n,m,s,t;

int dep[MAXN];//记录深度数组
int iter[MAXN];//当前弧数组 记录这个节点有效访问的第一条边的编号
int que[MAXN];

// 正反边从编号 2 开始配对，反向边编号为 e^1。
int cnt=2;
int head[MAXN];
int nxt[MAXM];
int to[MAXM];
int now[MAXM];//表示现在的流量
int cap[MAXM];//表示流量限制

void addedge(int u,int v,int w){
    nxt[cnt]=head[u];
    to[cnt]=v;
    cap[cnt]=w;
    now[cnt]=0;
    head[u]=cnt++;

    nxt[cnt]=head[v];
    to[cnt]=u;
    cap[cnt]=0;
    now[cnt]=0;
    head[v]=cnt++;
}

bool bfs(){
    // 只沿有剩余容量的边分层，汇点不可达时最大流已求完。
    for(int i=1;i<=n;i++){
        dep[i]=-1;
    }
    int ql=0,qr=0;
    dep[s]=0;
    que[qr++]=s;
    while(ql<qr){
        int u=que[ql++];
        for(int i=head[u];i;i=nxt[i]){
            int v=to[i];
            int w=cap[i];
            int k=now[i];
            if(dep[v]<0&&(k<w)){
                dep[v]=dep[u]+1;
                que[qr++]=v;
            }
        }
    }
    return dep[t]>=0;// 如果汇点未被访问到，说明无增广路径
}

int dfs(int u,int f){
```

```
    // 当前弧沿分层图送出最多 f；增广后同时更新正反残量。
    if(u==t){
        return f;
    }
    int flow=0;
    // 引用当前弧，避免重复扫描已耗尽的边。
    for(int &i=iter[u];i;i=nxt[i]){
        int v=to[i];
        int w=cap[i];
        int k=now[i];
        if(dep[u]+1==dep[v]&&k<w){
            int d=dfs(v,min(f,w-k));
            if(d>0){
                now[i]+=d;
                now[i^1]-=d;//更新反向边
                flow+=d;
                f-=d;
                if(f==0){
                    break;
                }
            }
        }
    }
    return flow;
}

int maxflow(){
    int flow=0;
    while(bfs()){
        for(int i=1;i<=n;i++){
            iter[i]=head[i];
        }
        int maxflow;
        while((maxflow=dfs(s,INF))>0){
            flow+=maxflow;
        }
    }
    return flow;
}

signed main()
{
    cin>>n>>m>>s>>t;
    if(s==t){
        cout<<0<<'\n';
        return 0;
    }
    for(int i=1;i<=m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        addedge(u,v,w);
    }
    cout<<maxflow()<<endl;
    return 0;
}
```

```cpp
// 用途：最小费用最大流。
// 时间：O(Fm log n)，F 为增广次数。
// 空间：O(n+m)。
// 例题：https://www.luogu.com.cn/problem/P3381
// 题意：在流量尽可能大的前提下使总费用最小。
// 思路：势能重标残量边费用，每轮 Dijkstra 找最小费用增广路。
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 5e3+5;
const int MAXM = 1e5+5;
const int INF = 1e18;

int n, m, s, t;
int mincost, maxflow;

// 势能数组（Johnson算法核心）
// 在第一次spfa之后，pot数组的含义是从起点s到达各个点的最小单位费用
int pot[MAXN];            // 用于调整边权，消除负权边影响

int dis[MAXN];
int pre[MAXN];            // 前驱边索引
bool vis[MAXN];

queue<int> q;
bool in[MAXN];

int head[MAXN];
int nxt[MAXM];
int to[MAXM];
int cap[MAXM];
int cost[MAXM];
int cnt = 2;

struct cmp {
    bool operator()(pair<int,int> a, pair<int,int> b) {
        return a.first > b.first;  // 按距离从小到大排序
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, cmp> heap;

inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while(ch < '0' || ch > '9') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

inline void addedge(int u, int v, int w, int c) {
    // 正向边：u->v
    nxt[cnt] = head[u];
    to[cnt] = v;
    cap[cnt] = w;
    cost[cnt] = c;
    head[u] = cnt++;

    // 反向边：v->u（容量为0，费用为负）
    nxt[cnt] = head[v];
    to[cnt] = u;
```

```
    cap[cnt] = 0;
    cost[cnt] = -c;
    head[v] = cnt++;
}

void spfa(int s) {
    // 初始化势能为无穷大
    for(int i = 1; i <= n; i++) {
        pot[i] = INF;
    }

    // 源点入队
    q.push(s);
    pot[s] = 0;
    in[s] = true;

    while(!q.empty()) {
        int u = q.front();
        q.pop();
        in[u] = false;

        // 遍历所有出边
        for(int i = head[u]; i; i = nxt[i]) {
            int v = to[i];
            int w = cap[i];     // 剩余容量
            int c = cost[i];    // 单位费用

            // 如果有容量且可以松弛
            if(w > 0 && pot[u] + c < pot[v]) {
                pot[v] = pot[u] + c;  // 更新势能

                // 如果节点v不在队列中，加入队列
                if(!in[v]) {
                    q.push(v);
                    in[v] = true;
                }
            }
        }
    }
}

// Dijkstra寻找增广路径
bool dijkstra() {
    for(int i = 1; i <= n; i++) {
        dis[i] = INF;
        pre[i] = -1;
        vis[i] = false;
    }

    dis[s] = 0;
    heap.push({0, s});

    while(!heap.empty()) {
        int u = heap.top().second;
        heap.pop();

        if(vis[u]) continue;
        vis[u] = true;

        for(int i = head[u]; i; i = nxt[i]) {
            int v = to[i];
            int residual = cap[i];
            int c_original = cost[i]; // 保存原始费用

            // 跳过容量为0的边
            if(residual <= 0) continue;
```

```

            // 使用势函数调整边权（关键步骤）
            int adjusted_cost = c_original + pot[u] - pot[v];

            // 如果可以松弛
            if(dis[u] + adjusted_cost < dis[v]) {
                dis[v] = dis[u] + adjusted_cost;
                pre[v] = i;  // 记录前驱边索引
                heap.push({dis[v], v});
            }
        }
    }
    return dis[t] < INF;
}

// 最小费用最大流主函数
void mincostmaxflow() {
    spfa(s);
    maxflow = 0;
    mincost = 0;

    while(dijkstra()) {
        // 计算增广路径上的最小容量
        int flow = INF;
        for(int u = t; u != s; u = to[pre[u] ^ 1]) {
            flow = min(flow, cap[pre[u]]);
        }

        //  更新流量和费用
        maxflow += flow;
        // 实际费用 = 调整后距离 + 汇点势能
        mincost += flow * (dis[t] + pot[t]);

        // 更新路径上的边容量
        for(int u = t; u != s; u = to[pre[u] ^ 1]) {
            cap[pre[u]] -= flow;      // 正向边减流量
            cap[pre[u] ^ 1] += flow;  // 反向边加流量
        }

        //  更新势函数（为下一轮准备）
        for(int i = 1; i <= n; i++) {
            if(dis[i] < INF) {
                pot[i] += dis[i];  // 关键：更新势能
            }
        }
    }
}

signed main()
{
    // 读入节点数、边数、源点、汇点
    n = read(), m = read(), s = read(), t = read();

    // 读入并添加所有边
    for(int i = 1; i <= m; i++) {
        int u, v, w, c;
        u = read(), v = read(), w = read(), c = read();
        addedge(u, v, w, c);
    }

    // 计算最小费用最大流
    mincostmaxflow();

    // 输出最大流量和最小费用
    cout << maxflow << ' ' << mincost << endl;

```

```
199       return 0;
200   }
```

```cpp
// 用途：每条有向边有流量下界和上界时，求源点到汇点的最大可行流。
// 时间：两次 Dinic，最坏 O(n□m)。
// 空间：O(n+m)。
// 注意：源汇不同，0≤下界≤上界，流量总和不超过 2×10^18；无解输出 -1。
// 例题：https://loj.ac/p/116
// 题意：每条边有流量上下界，求 s 到 t 的最大可行流。
// 思路：先按下界平衡各点流量并求可行流，再在残量网络中增广 s→t。
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 205, MAXM = 10005;
const int MAXE = 2 * (MAXM + MAXN + 5);
const long long INF = LLONG_MAX / 4;
int head[MAXN], to[MAXE], nxt[MAXE], edgeCount;
int level[MAXN], current[MAXN], que[MAXN];
int superEdges[MAXN * 2], superCount;
long long capacity[MAXE], balance[MAXN];

int addEdge(int u, int v, long long cap) {
    int forward = edgeCount;
    to[edgeCount] = v;
    capacity[edgeCount] = cap;
    nxt[edgeCount] = head[u];
    head[u] = edgeCount++;
    to[edgeCount] = u;
    capacity[edgeCount] = 0;
    nxt[edgeCount] = head[v];
    head[v] = edgeCount++;
    return forward;
}

bool bfs(int source, int sink, int vertices) {
    fill(level, level + vertices + 1, -1);
    int left = 0, right = 0;
    que[right++] = source;
    level[source] = 0;
    while (left < right) {
        int u = que[left++];
        for (int e = head[u]; e != -1; e = nxt[e]) {
            int v = to[e];
            if (capacity[e] > 0 && level[v] == -1) {
                level[v] = level[u] + 1;
                que[right++] = v;
            }
        }
    }
    return level[sink] != -1;
}

long long dfs(int u, int sink, long long pushed) {
    if (u == sink || pushed == 0) return pushed;
    for (int &e = current[u]; e != -1; e = nxt[e]) {
        int v = to[e];
        if (capacity[e] == 0 || level[v] != level[u] + 1) continue;
        long long flow = dfs(v, sink, min(pushed, capacity[e]));
        if (flow == 0) continue;
        capacity[e] -= flow;
        capacity[e ^ 1] += flow;
        return flow;
    }
    return 0;
}

long long maxflow(int source, int sink, int vertices) {
    long long total = 0;
    while (bfs(source, sink, vertices)) {
```

```
        copy(head, head + vertices + 1, current);
        while (long long flow = dfs(source, sink, INF)) total += flow;
    }
    return total;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    fill(head, head + MAXN, -1);
    int n, m, source, sink;
    cin >> n >> m >> source >> sink;
    int superSource = n + 1, superSink = n + 2;
    for (int i = 0, u, v; i < m; ++i) {
        // 先固定每条边的下界，只把可追加的 upper-lower 留在残量网络。
        long long lower, upper;
        cin >> u >> v >> lower >> upper;
        addEdge(u, v, upper - lower);
        balance[u] -= lower;
        balance[v] += lower;
    }
    int returnEdge = addEdge(sink, source, INF);
    // 加 t→s 后变无源汇可行循环；超级源汇补齐每点的流量差。
    long long required = 0;
    for (int u = 1; u <= n; ++u) {
        if (balance[u] > 0) {
            superEdges[superCount++] = addEdge(superSource, u, balance[u]);
            required += balance[u];
        } else if (balance[u] < 0) {
            superEdges[superCount++] = addEdge(u, superSink, -balance[u]);
        }
    }
    if (maxflow(superSource, superSink, superSink) != required) {
        cout << -1 << '\n';
        return 0;
    }
    long long baseFlow = capacity[returnEdge ^ 1];
    // 可行流确定后，移除辅助边，再在原网络的残量图上增广。
    capacity[returnEdge] = capacity[returnEdge ^ 1] = 0;
    for (int i = 0; i < superCount; ++i)
        capacity[superEdges[i]] = capacity[superEdges[i] ^ 1] = 0;
    cout << baseFlow + maxflow(source, sink, superSink) << '\n';
    return 0;
}
```

```cpp
// 用途：判断无源汇的有向网络是否存在满足每条边上下界的循环流，并输出各边流量。
// 时间：一次 Dinic，最坏 O(n□m)。
// 空间：O(n+m)。
// 注意：0≤下界≤上界；所有顶点须满足流入量=流出量；原边流量=下界+残量网络流量。
// 例题：https://loj.ac/p/115
// 题意：每条边有流量范围，求是否存在可行循环流；有则按输入顺序输出每条边的流量。
// 思路：先固定下界，用超级源汇补齐各点流量差；附加源边全部满流即有解。
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 205;
const int MAXM = 10205;
const int MAXV = 205;
const int MAXE = MAXM*2+MAXN*2;
const long long INF = LLONG_MAX / 4;

int n,m,s,t;
long long in[MAXN];

struct Edge{
    int u,v;
    long long lower,higher;
};
Edge edge[MAXM];
int edgeid[MAXM];

int dep[MAXV];
int iter[MAXV];

int cnt=2;
int head[MAXV];
int nxt[MAXE];
int to[MAXE];
long long now[MAXE];
long long cap[MAXE];

inline long long read(){
    long long x=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9'){
        if(ch=='-')
            f=-1;
        ch=getchar();
    }
    while(ch>='0' && ch<='9')
        x=x*10+ch-'0',ch=getchar();
    return x*f;
}

void addedge(int u,int v,long long w){
    nxt[cnt]=head[u];
    to[cnt]=v;
    cap[cnt]=w;
    now[cnt]=0;
    head[u]=cnt++;

    nxt[cnt]=head[v];
    to[cnt]=u;
    cap[cnt]=0;
    now[cnt]=0;
    head[v]=cnt++;
}

bool bfs(){
    for(int i=1;i<=t;i++){
        dep[i]=-1;
    }
```

```
    queue<int>q;
    dep[s]=0;
    q.push(s);
    while(!q.empty()){
        int u=q.front();
        q.pop();
        for(int i=head[u];i;i=nxt[i]){
            int v=to[i];
            long long w=cap[i];
            long long k=now[i];
            if(dep[v]<0&&(k<w)){
                dep[v]=dep[u]+1;
                q.push(v);
            }
        }
    }
    return dep[t]>=0;
}

long long dfs(int u,long long f){
    if(u==t){
        return f;
    }
    long long flow=0;
    for(int &i=iter[u];i;i=nxt[i]){
        int v=to[i];
        long long w=cap[i];
        long long k=now[i];
        if(dep[u]+1==dep[v]&&k<w){
            long long d=dfs(v,min(f,w-k));
            if(d>0){
                now[i]+=d;
                now[i^1]-=d;
                flow+=d;
                f-=d;
                if(f==0){
                    break;
                }
            }
        }
    }
    return flow;
}

long long maxflow(){
    long long flow=0;
    while(bfs()){
        for(int i=1;i<=t;i++){
            iter[i]=head[i];
        }
        long long pushed;
        while((pushed=dfs(s,INF))>0){
            flow+=pushed;
        }
    }
    return flow;
}

int main()
{
    n=read(),m=read();
    s=n+1,t=s+1;
    for(int i=1;i<=m;i++){
        edge[i].u=read(),edge[i].v=read(),edge[i].lower=read(),edge[i].higher=read();
    }
    for(int i=1;i<=m;i++){
```

```
        int u=edge[i].u;
        int v=edge[i].v;
        long long lower=edge[i].lower;
        long long higher=edge[i].higher;
        edgeid[i]=cnt;

        addedge(u,v,higher-lower);

        in[u]-=lower;
        in[v]+=lower;
    }

    // 添加附加源汇的边  并记录附加边总权值
    long long need=0;
    for(int i=1;i<=n;i++){
        if(in[i]>0){
            addedge(s,i,in[i]);
            need+=in[i];
        }
        else{
            addedge(i,t,-in[i]);
        }
    }

    long long flow=maxflow();

    if(flow!=need){
        cout<<"NO"<<endl;
    }
    else{
        cout<<"YES"<<endl;
        for(int i=1;i<=m;i++){
            long long actual=now[edgeid[i]]+edge[i].lower;
            cout<<actual<<endl;
        }
    }
    return 0;
}
```

```cpp
// 用途：求无向图中至少删除多少个非端点，才能使指定两点不连通（最小点割）。
// 时间：Dinic 最坏 O(n□m)。
// 空间：O(n+m)。
// 注意：题目保证两个端点不直接相连；不能删除的端点拆点容量设为 INF。
// 例题：https://www.luogu.com.cn/problem/P1345
// 题意：删去尽量少的中间电脑，使两台指定电脑无法通信。
// 思路：每点拆成入点和出点，中间点连容量 1 的边；原图边连 INF，最大流即最小点割。
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int MAXN = 205;//这里是2*n
const int MAXM = 3000;//这里是 2*n+4*m
const int INF = 1e18;

int n,m,s,t;

int dep[MAXN];
int iter[MAXN];
int que[MAXN];

int head[MAXN];
int nxt[MAXM];
int to[MAXM];
int now[MAXM];
int cap[MAXM];
int cnt=2;

void addedge(int u,int v,int w){
    nxt[cnt]=head[u];
    to[cnt]=v;
    cap[cnt]=w;
    head[u]=cnt++;
}

//BFS构建分层图，并判断是否存在增广路径
bool bfs(){
    for(int i=1;i<=2*n;i++){
        dep[i]=-1;
    }
    int left=0,right=0;
    dep[s]=0;
    que[right++]=s;
    while(left<right){
        int u=que[left++];
        for(int i=head[u];i;i=nxt[i]){
            int v=to[i];
            int w=cap[i];
            int k=now[i];
            if(dep[v]<0&&(k<w)){
                dep[v]=dep[u]+1;
                que[right++]=v;
            }
        }
    }
    return dep[t]>=0;// 如果汇点未被访问到，说明无增广路径
}

// DFS寻找增广路径（多路增广）
//多路增广的含义是来到了一个节点 不仅仅是只去一条边增广  而是去多条边一起增广
int dfs(int u,int f){
    //表示当前来到了u节点 有f的流量可供使用  f就是目前这条路最多可以消耗掉的流量
    if(u==t){
        return f;
    }
    int flow=0;
    for(int &i=iter[u];i;i=nxt[i]){
```

```
        //注意这里是引用  iter会随着i发生变化
        int v=to[i];
        int w=cap[i];
        int k=now[i];
        if(dep[u]+1==dep[v]&&k<w){
            int d=dfs(v,min(f,w-k));
            if(d>0){
                now[i]+=d;
                now[i^1]-=d;//更新反向边
                flow+=d;
                f-=d;
                if(f==0){
                    break;
                }
            }
        }
    }
    return flow;
}

// Dinic算法主函数
int maxflow(){
    int flow=0;
    while(bfs()){
        //当前弧全部初始化为最初值
        for(int i=1;i<=2*n;i++){
            iter[i]=head[i];
        }
        int maxflow;
        while((maxflow=dfs(s,INF))>0){
            flow+=maxflow;
        }
    }
    return flow;
}

signed main()
{
    cin>>n>>m>>s>>t;
    for(int i=1;i<=n;i++){
        if(i==s||i==t){
            addedge(i,i+n,INF);
            addedge(i+n,i,0);//反边
        }
        else{
            addedge(i,i+n,1);
            addedge(i+n,i,0);//反边
        }
    }
    for(int i=1;i<=m;i++){
        int u,v;
        cin>>u>>v;
        addedge(u+n,v,INF);
        addedge(v,u+n,0);//反边

        addedge(v+n,u,INF);
        addedge(u,v+n,0);//反边
    }
    cout<<maxflow()<<endl;
    return 0;
}
```

```cpp
// 用途：求有向图的强连通分量，按最小点编号顺序输出。
// 时间：O(n+m)。
// 空间：O(n+m)。
// 注意：n<=1e4，m<=1e5；递归 DFS。
// 例题：https://www.luogu.com.cn/problem/B3609
// 题意：把有向图划分成互相可达的极大点集。
// 思路：DFS 求 dfn/low，回溯到强连通分量根时弹出栈中节点。
#include <bits/stdc++.h>

using namespace std;

const int MAXN = 10001;
const int MAXM = 100001;
int n, m;

int head[MAXN];
int nxt[MAXM];
int to[MAXM];
int cntg;

// dfn 是首次访问序，low 是能回到的最早祖先序。
int dfn[MAXN];
int low[MAXN];
int cntd;

int sta[MAXN];
int top;

int belong[MAXN];
int sccArr[MAXN];
int sccl[MAXN];
int sccr[MAXN];
int idx;
int sccCnt;

bool sccPrint[MAXN];

void addEdge(int u, int v) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    head[u] = cntg;
}

void tarjan(int u) {
    dfn[u] = low[u] = ++cntd;
    sta[++top] = u;
    for (int e = head[u]; e > 0; e = nxt[e]) {
        int v = to[e];
        if (dfn[v] == 0) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else {
            if (belong[v] == 0) {
                low[u] = min(low[u], dfn[v]);
            }
        }
    }
    // 当 low 等于 dfn 时，弹出一个完整强连通分量。
    if (dfn[u] == low[u]) {
        sccCnt++;
        sccl[sccCnt] = idx + 1;
        int pop;
        do {
            pop = sta[top--];
            belong[pop] = sccCnt;
            sccArr[++idx] = pop;
```

```
        } while (pop != u);
        sccr[sccCnt] = idx;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1, u, v; i <= m; i++) {
        cin >> u >> v;
        addEdge(u, v);
    }
    for (int i = 1; i <= n; i++) {
        if (dfn[i] == 0) {
            tarjan(i);
        }
    }
    cout << sccCnt << "\n";
    for (int i = 1; i <= sccCnt; i++) {
        sort(sccArr + sccl[i], sccArr + sccr[i] + 1);
    }
    for (int i = 1; i <= n; i++) {
        int scc = belong[i];
        if (!sccPrint[scc]) {
            sccPrint[scc] = true;
            for (int j = sccl[scc]; j <= sccr[scc]; j++) {
                cout << sccArr[j] << " ";
            }
            cout << "\n";
        }
    }
    return 0;
}
```

```
// 边双连通分量模版题，C++版
// 给定一张无向图，一共n个点、m条边
// 图中可能存在多个连通区，对每个连通区求边双连通分量
// 先打印边双连通分量的总数量，然后对每个边双连通分量
// 打印节点个数，然后任意顺序打印该边双连通分量的节点
// 请保证原图即使有重边和自环，答案依然正确
// 1 <= n <= 5 * 10^5
// 1 <= m <= 2 * 10^6
// 测试链接 : https://www.luogu.com.cn/problem/P8436
// 如下实现是C++的版本，C++版本和java版本逻辑完全一样
// 提交如下代码，可以通过所有测试用例

#include <bits/stdc++.h>

using namespace std;

const int MAXN = 500001;
const int MAXM = 2000001;
int n, m;

int head[MAXN];
int nxt[MAXM << 1];
int to[MAXM << 1];
int cntg;

int dfn[MAXN];
int low[MAXN];
int cntd;

int sta[MAXN];
int top;

int ebccSiz[MAXN];
int ebccArr[MAXN];
int ebccl[MAXN];
int ebccr[MAXN];
int idx;
int ebccCnt;

void addEdge(int u, int v) {
   nxt[++cntg] = head[u];
   to[cntg] = v;
   head[u] = cntg;
}

void tarjan(int u, int preEdge) {
    dfn[u] = low[u] = ++cntd;
    sta[++top] = u;
    for (int e = head[u]; e > 0; e = nxt[e]) {
        if ((e ^ 1) == preEdge) {
            continue;
        }
        int v = to[e];
        if (dfn[v] == 0) {
            tarjan(v, e);
            low[u] = min(low[u], low[v]);
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) {
        ebccCnt++;
        ebccSiz[ebccCnt] = 0;
        ebccl[ebccCnt] = idx + 1;
        int pop;
        do {
```

```
            pop = sta[top--];
            ebccSiz[ebccCnt]++;
            ebccArr[++idx] = pop;
        } while (pop != u);
        ebccr[ebccCnt] = idx;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cntg = 1;
    cin >> n >> m;
    for (int i = 1, u, v; i <= m; i++) {
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    for (int i = 1; i <= n; i++) {
        if (dfn[i] == 0) {
            tarjan(i, 0);
        }
    }
    cout << ebccCnt << "\n";
    for (int i = 1; i <= ebccCnt; i++) {
        cout << ebccSiz[i] << "\n";
        for (int j = ebccl[i]; j <= ebccr[i]; j++) {
            cout << ebccArr[j] << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

```
// 点双连通分量模版题1，C++版
// 给定一张无向图，一共n个点、m条边，建图时忽略自环
// 打印点双连通分量的个数
// 打印每个点双连通分量的大小和节点编号
// 打印顺序随意
// 每个孤立点分配一个点双连通分量
// 1 <= n <= 5 * 10^5
// 1 <= m <= 2 * 10^6
// 测试链接 : https://www.luogu.com.cn/problem/P8435
// 如下实现是C++的版本，C++版本和java版本逻辑完全一样
// 提交如下代码，可以通过所有测试用例

#include <bits/stdc++.h>

using namespace std;

const int MAXN = 500001;
const int MAXM = 2000001;
int n, m;

int head[MAXN];
int nxt[MAXM << 1];
int to[MAXM << 1];
int cntg;

int dfn[MAXN];
int low[MAXN];
int cntd;

int sta[MAXN];
int top;

int vbccSiz[MAXN];
int vbccArr[MAXN << 1];
int vbccl[MAXN];
int vbccr[MAXN];
int idx;
int vbccCnt;

void addEdge(int u, int v) {
    nxt[++cntg] = head[u];
    to[cntg] = v;
    head[u] = cntg;
}

void tarjan(int u) {
    dfn[u] = low[u] = ++cntd;
    sta[++top] = u;
    for (int e = head[u]; e > 0; e = nxt[e]) {
        int v = to[e];
        if (dfn[v] == 0) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u]) {
                vbccCnt++;
                vbccSiz[vbccCnt] = 1;
                vbccArr[++idx] = u;
                vbccl[vbccCnt] = idx;
                int pop;
                do {
                    pop = sta[top--];
                    vbccSiz[vbccCnt]++;
                    vbccArr[++idx] = pop;
                } while (pop != v);
                vbccr[vbccCnt] = idx;
            }
```

```
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1, u, v; i <= m; i++) {
        cin >> u >> v;
        if (u != v) {
            addEdge(u, v);
            addEdge(v, u);
        }
    }
    for (int i = 1; i <= n; i++) {
        if (dfn[i] == 0) {
            if (head[i] == 0) {
                vbccCnt++;
                vbccSiz[vbccCnt] = 1;
                vbccArr[++idx] = i;
                vbccl[vbccCnt] = vbccr[vbccCnt] = idx;
            } else {
                tarjan(i);
            }
        }
    }
    cout << vbccCnt << "\n";
    for (int i = 1; i <= vbccCnt; i++) {
        cout << vbccSiz[i] << "\n";
        for (int j = vbccl[i]; j <= vbccr[i]; j++) {
            cout << vbccArr[j] << " ";
        }
        cout << "\n";
    }
    return 0;
}
```